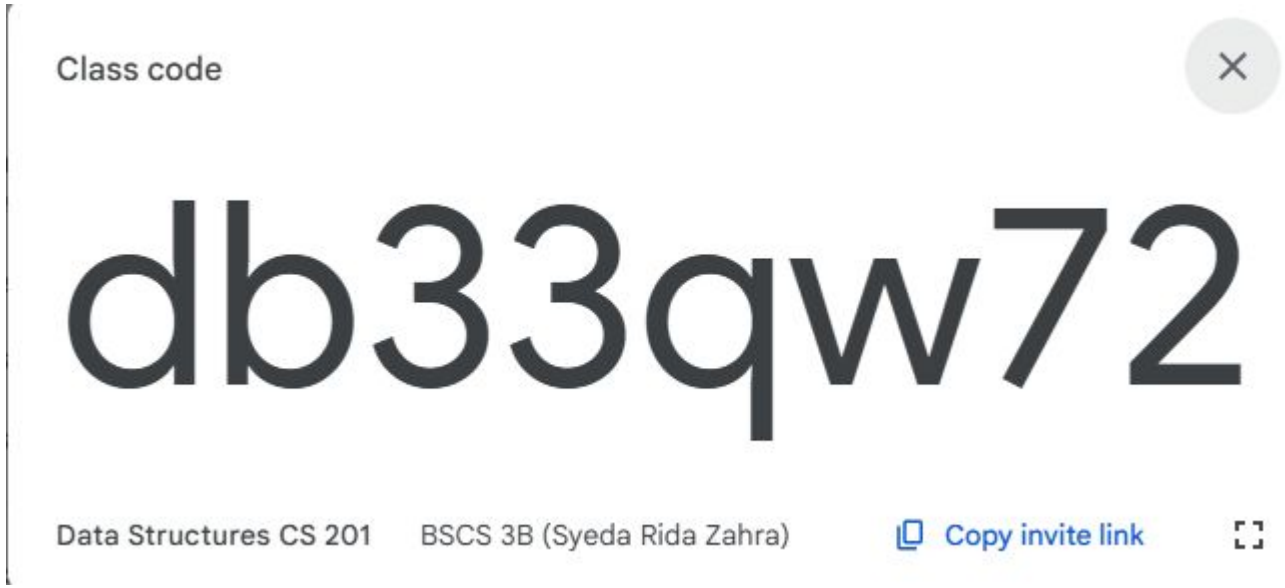
Data Structures & Algorithm

INSTRUCTOR: RIDA ZAHRA

DEPARTMENT OF COMPUTER SCIENCE

Google Classroom Codes

Theory Code



Evaluation Criteria

Assessment	%Age
Assignments/ Quizzes	7.5% + 11.25% = 18.75%
Viva/Sessional	10% + 15% = 25%
Mid Term Exam	22.5%
Final Term Exam	33.75%
Total	100

Assignments	Quizzes
Assignments will be due at the beginning of the class. Under normal circumstances, late work will not be accepted. Students are expected to do their own assignments. Plagiarism will be observed strictly.	• Surprise/ Announced quizzes to assess the understanding of taught topics.

- Revise the topic covered in class the same day.
- Complete the tasks given, you might get bonus points for that. ☺
- Feel free to consult me during my office hours.

Few Things to Remember

- Show sensibility in your conduct.
- Learning should be your primary objective.
- Attendance will be marked within 15 minutes at the start of the class.
- 80% of your attendance is mandatory.
- **Zero tolerance policy** on attendance, discipline in class during lectures.
- Assignments must be submitted on time, no late submissions.
- In case of copied assignment both parties will be given **zero.**
- Don't miss your Classes, Quizzes, Presentations, Assignments and Projects/Presentations.
- You all need to be a good human being.

Course Objectives

- To understand the design of fundamental data structures as well as algorithms that operate on them
- To understand the fundamental tradeoffs in the design of the data structures
- To introduce tools for analyzing the time and space complexity of data structures

Course Learning Outcomes		
After successful completion of this course, the students should be able to:	GA	BT Level
1. **Explain** the fundamentals of data structure as well as algorithms that operate on them	2	C2
2. **Apply** and design various data structures methods for specified applications.	4	C3
3. **Analyze** empirically the behavior of algorithms in terms of time andmemory used	3	C4

Mapping of CLO's to GA's

GA's/CLOs	CLO1	CLO2	CLO3
GA1 (Academic Education)			
GA2 (Knowledge for Solving Computing Problems)	✔		
GA3 (Problem Analysis)			✔
GA4 (Design/Development of Solutions)		✔	
GA5 (Modern Tool Usage)			
GA6 (Individual and Teamwork)			
GA7 (Communication)			
GA8 (Computing Professionalism and Society)			
GA9 (Ethics)			
GA10 (Life-long Learning)			

Text and Reference Books

Text Books:

- Data Structures and Algorithms in C++ by Michael T. Goodrich, Roberto Tamassia, and David Mount (2nd Edition)
- Data Structures and Algorithm Analysis in C++ by Mark Allen Weiss (4th Edition)

Reference Books

- Malik, D S. Data Structures in C++, 2nd Edition, Cengage Learning

Course Overview

- Introduction to data structures.
- Analysis tools.
- Arrays.
- Linked list.
- Stacks.
- Queues.
- Trees.
- Binary search tree.
- Hash tables.
- Sorting.
- Graphs

Prerequisites

This module assumes that you understand the fundamentals of Programming

- Variables, statements, functions, loops, etc.

Note: Basic structure of C++ will be used in this subject.

Programming Basics

Today’s Goal

To review basic programming concepts like:

- Variables and constants.
- Data types.
- Operators.
- Control flow.

Variable and Constant

In a program, **data values** can be **constant** or **variable**.

If values are variable they can be changed by the program and the user.

When a program is running, the data values are held in **memory** while they are being worked on.

Constant

Constants are values which cannot be modified e.g. the value of Pi

To declare a constant in Java, we write a keyword “final” before the variable type.

```
final double pi = 3.14;
```

Variable

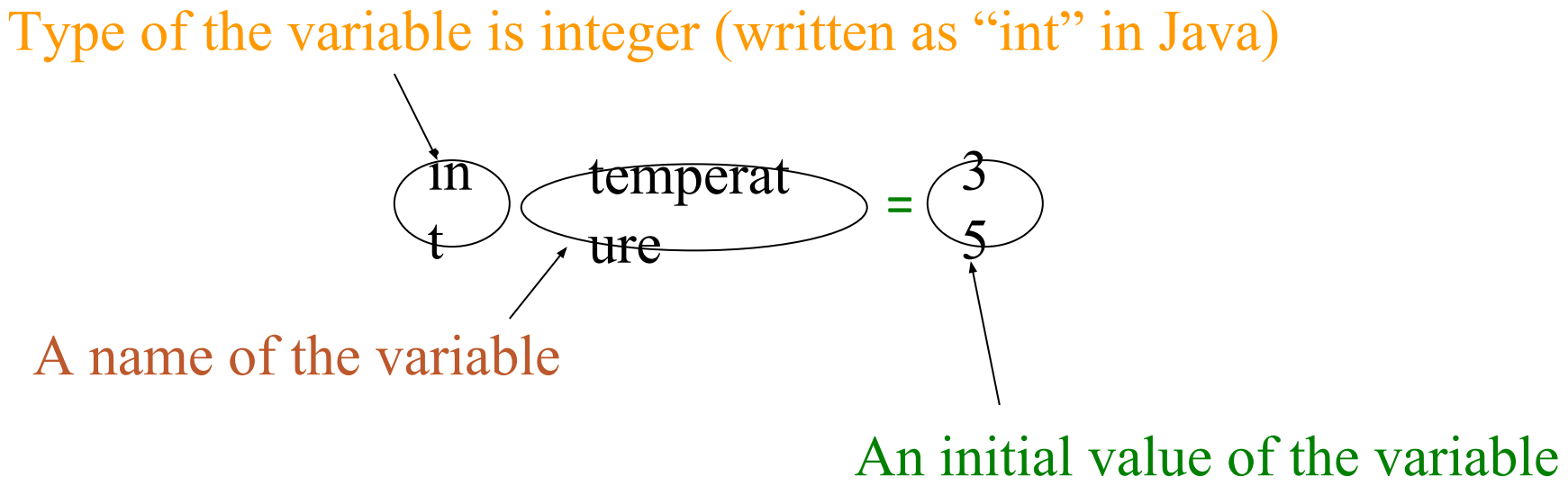
- To store a value inside a computer a ‘variable’ is used.
- A variable is a space in the memory to store a value.
- This space is reserved until the variable is required.

What Makes a Variable?

Variable has three important characteristics:

- Type
  - How much memory do a variable need.
    - This information is determined by a type.
- Name
  - How to differentiate a variable with another variable of the same type.
    - Name refers to the memory location assigned to this variable.
- Value
  - What is the value?
    - The actual value contained by a variable.

An Example of a Variable



Rules for naming a variable

Following are the rules for naming variables:

- Variable names in C++ can range from 1 to 255 characters.
- All variable names must begin with a letter of the alphabet or an underscore(_).
- After the first initial letter, variable names can also contain letters and numbers.
- Variable names are case sensitive.
- No spaces or special characters are allowed.
- You cannot use a C++ keyword (a reserved word) as a variable name.

Some Examples

Here are some examples of acceptable variable names:

Mohd, Ali, abc, move_name, a_123, myname50, _temp, j, a23b9, retVal

Type of a Variable-Data type

Computer programs use data types to organize different types of data in a program.

Among other advantages a ‘type’ binds the memory to a variable name.

Following data types exist:

- Int
- Float
- Double
- Char
- Boolean

Int

The type int is of **4 bytes** (8x4= 32 bits).

Therefore, it can hold maximum of 2,147,483,647 value.

It can also hold values in negative down to

-2,147,483,648.

Data type for real numbers-float

int cannot hold a real value.

Therefore, the type “float” and “double” are used to hold real values.

A variable of type float holds floating type data or real numbers, e.g., -11.4, 3.1416.

Float **occupies 4 bytes** in memory.

It can store values in range 3.4 x10^-38 to 3.4x10^38

Data type for real numbers-double

Double takes 8 bytes (8x8=64 bits) of memory instead of 4 bytes of an int.

Out of the 8 bytes in a double 4 bytes are used to hold the value before the decimal point and 4 bytes for the value after the decimal point.

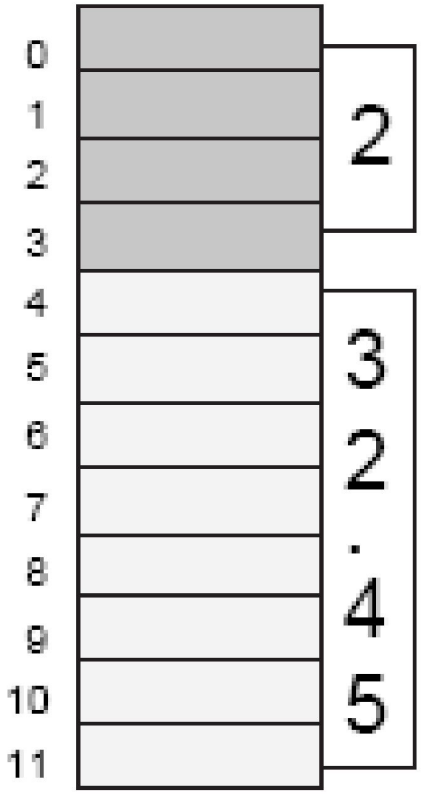
Relative Comparison of int and double

int numPeople = 2;

Reserves 32 bits (4 bytes)
and sets the value stored
in that space to 2. The name
‘numPeople’ is associated with
this space.

double bill = 32.45;

Reserves 64 bits (8 bytes)
and sets the value stored
in that space to 32.45. The name
‘bill’ is associated with
this space.



Char Data Type

A variable of type char holds only a single lower case letter, a single uppercase letter, a single digit or a single special character (e.g., $ or *).

char takes only 1 byte (8 bits) in memory.

It can store values from -128 to 127.

Collection of characters=> string

Boolean Data Type

Java has a simple type, called boolean, for logical values.

It can have only one of two possible values, true or false.

This is the type returned by all relational operators, suchas **a < b**.

boolean is also the type *required by the conditional expressions that govern the* control statements such as if and for.

Variable declarations

Variable in a computer program is declared as:

datatype variable_name;

For example: int var;

float var;

Variables can also be assigned values as:
datatype variable_name= value;
For example: int var=8;
float var=4.5;
double var=23.4556;
char var=‘a’;
string str1 = "Hello";

Operators used in Computer Programming

- Assignment

A binary operator '=':

variable = expression;

e.g. x=x+1

- Arithmetic

Used to compute arithmetic expressions e.g. '+', '-', '*', '/', '%'.

Increment/ Decrement Operators

Two unary operators:

1) ++ increments its operand by 1
2) -- decrements its operand by 1

The operand must be a numerical variable.

Each operation can appear in two versions:

- **prefix** version evaluates the value of the operand after performing the increment/decrement operation
- **postfix** version evaluates the value of the operand before performing the increment/decrement operation

Relational Operator

Relational operators determine the relationship that one operand has to the other operand, specifically equality and ordering.

The outcome is always a value of type **boolean**.

They are most often used in branching and loop control statements.

e.g. ‘==’, ‘!=’, ‘>’, ‘<’, ‘>=’, ‘<=’

Logical Operators

Logical operators act upon boolean operands only.

The outcome is always a value of type boolean.

E.g. AND(&), OR(|), NOT(!), XOR(^).

Control Flow

Writing a program means typing statements into a file.

Without control flow, the interpreter would execute these statements in the order they appear in the file, left-to-right, top-down.

Control flow statements, when inserted into the text of the program, determine in which order the program should be executed.

Types of control flow statements

Selection statements
- if
- if-else
- if-else-if
- switch

Iteration Statements
- While
- Do-while
- For

Jump Statements
- break
- return
- continue

Simple/Compound if statements

The component statement may be:

- simple
  - if (expression) statement;
- compound
  - if (expression) {
    statement;
    }

If-else statement

Suppose you want to perform two different statements depending on the outcome of a boolean expression. if-else statement can be used.

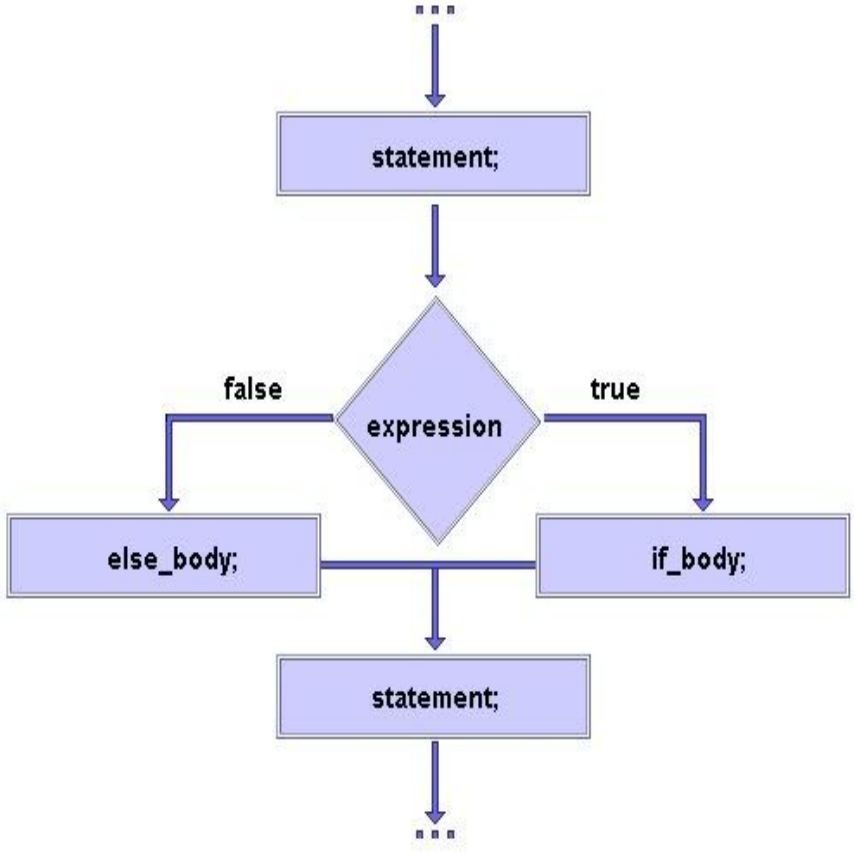
General form:

- if (expression) statement1
  else statement2

Again, statement1 and

statement2 may be simple

or compound.



If-else-if statement

General form:

- if (expression1)
  statement1
  else if (expression2)
  statement2
  else if (expression3)
  statement3
  …
  else statement

Example

Write a program that inputs test score of a student and displays his grade according to the following criteria

Test Score	Grade
>=90	A
80-89	B
70-79	C
60-69	D
Below 60	F

Example

```
{
int marks;
cout<<“Enter marks:”;
cin>>marks;
        if(marks>=90)
                Cout<<“Grade A”;
        else if(marks>=80)
                Cout<<“Grade B”;
        else if(marks>=70)
                Cout<<“Grade C”;
        else if(marks>=60)
                Cout<<“Grade D”;
        else
                Cout<<“Grade F”;
}
```

Switch statement

switch provides a better alternative than if-else-if when the execution follows several branches depending on the value of an expression.

General form:

```
switch (expression) {
case value1: statement1; break;
case value2: statement2; break;
case value3: statement3; break;
…
default: statement;
}
```

Example

Write a program that inputs a character from the user and checks whether it is a vowel or consonant.

Example

```
int main()
{
char c;
cout<<“Enter an alphabet:”;
cin>>c;
switch(c)
{
        case ‘a’:
        case ’A’:
                cout<<“You entered vowel”;
        break;
        case ‘e’:
        case ’E’:
                cout<<“You entered vowel”;
        break;
        case ‘i’:
        case ’I’:
                cout<<“You entered vowel”;
        break;
        case ‘o’:
        case ’O’:
                cout<<“You entered vowel”;
        break;
        case ‘u’:
        case ’U’:
                cout<<“You entered vowel”;
        break;
default:
        cout<<“You enter consonant.”;
}
}
```

while

General form:

- while (*expression) statement*

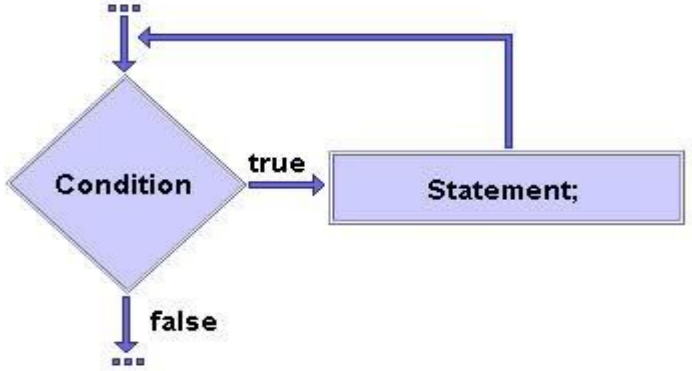
where expression must be of type boolean.

The component statement may be:

- simple
  - while (expression) statement;
- compound
  - while (expression) {

    statement;

    }



Example

Write a program that displays first five numbers and their sum using while loop.

Example

```
int main()
{
    int c, sum;
    c=1;
    sum=0;
    while(c<=5)
    {
        cout<<c<<endl;
        sum = sum + c;
        c=c+1;
    }
cout<<“Sum is = ”<<sum;
}
```

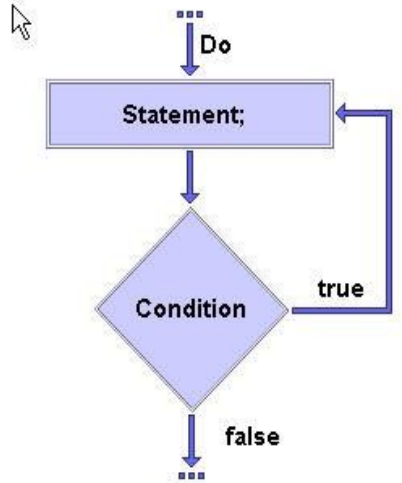
Do-while statement

If a component statement has to be executed at least once, the do-while statement is more appropriate than the while statement.

General form:

- do statement
  while (*expression);*

where expression must be of type boolean.



Example

Write a program that gets starting and ending point from the user and displays all odd numbers in the given range using do-while loop

Example

```
{
int c, s, e;
cout<<“Enter starting number:”;
cin>>s;
cout<<“Enter ending number:”;
cin>>e;
c = s;
do
{
      if(c % 2 !=0)
            cout<<c<<endl;
      c = c + 1;
}
while(c<=e);
}
```

For statement

When iterating over a range of values, ‘for’ statement is more suitable to use than while or do-while.

General form:

- for (initialization; termination; increment)

  statement

where:

- initialization statement is executed once before the first iteration
- termination expression is evaluated before each iteration to determine when the loop should terminate
- increment statement is executed after each iteration

Example

Product of all odd numbers from 1 to 10

```
int main()
{
        int product = 1;
        int c;
        for( c=1; c<=10; c = c+2)
                product *=c;
        cout<<“Result is”<<product;
}
```

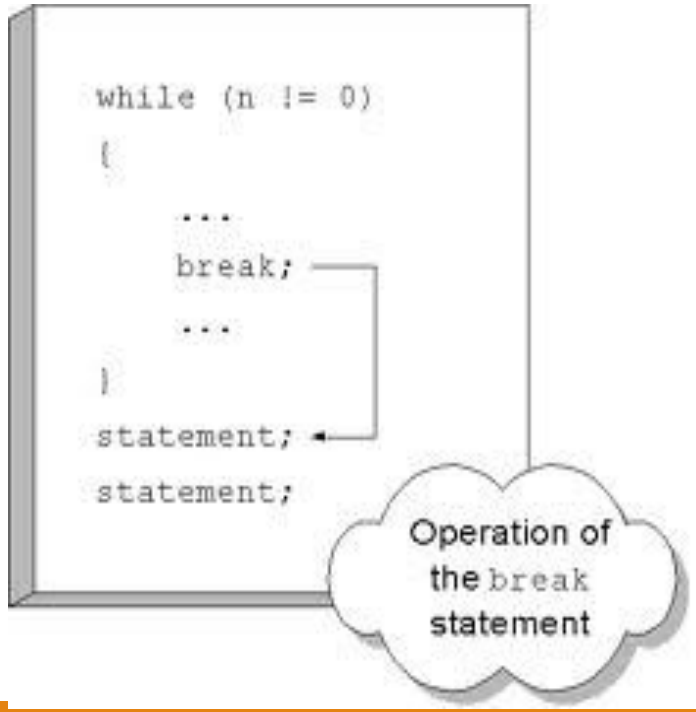
Break Statement

The break statement has three uses:

1. to terminate a case inside the switch statement
2. to exit an iterative statement
3. to transfer control to another statement

Other statements after the break are not executed.

The control directly moves outside the body and the statement that comes after the body is executed.



```
for(int x = 1; x<=5; x++)
{
        cout<<"Questioning"<<endl;
        break;
        cout<<"Gateway to knowledge";
}
cout<<"Bye";
```

Return statement

The return statement is used to return from the current method: it causes program control to transfer back to the caller of the method.

The return statement returns the flow of the execution to the function from where it is called.

This statement does not mandatorily need any conditional statements. As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called.

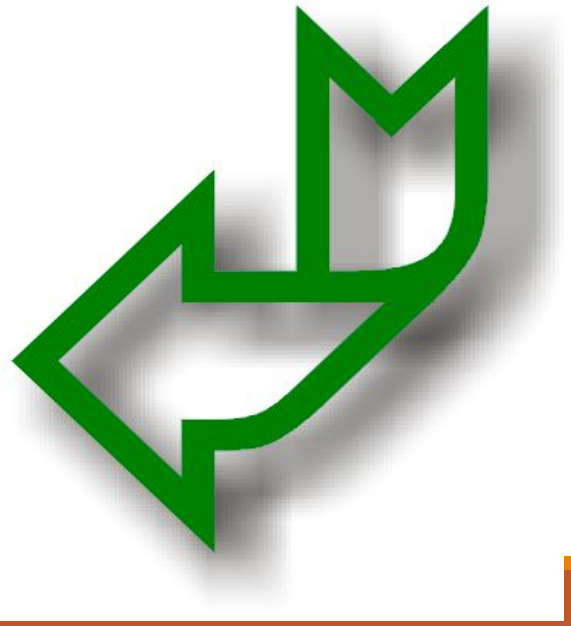
The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.

Two forms:

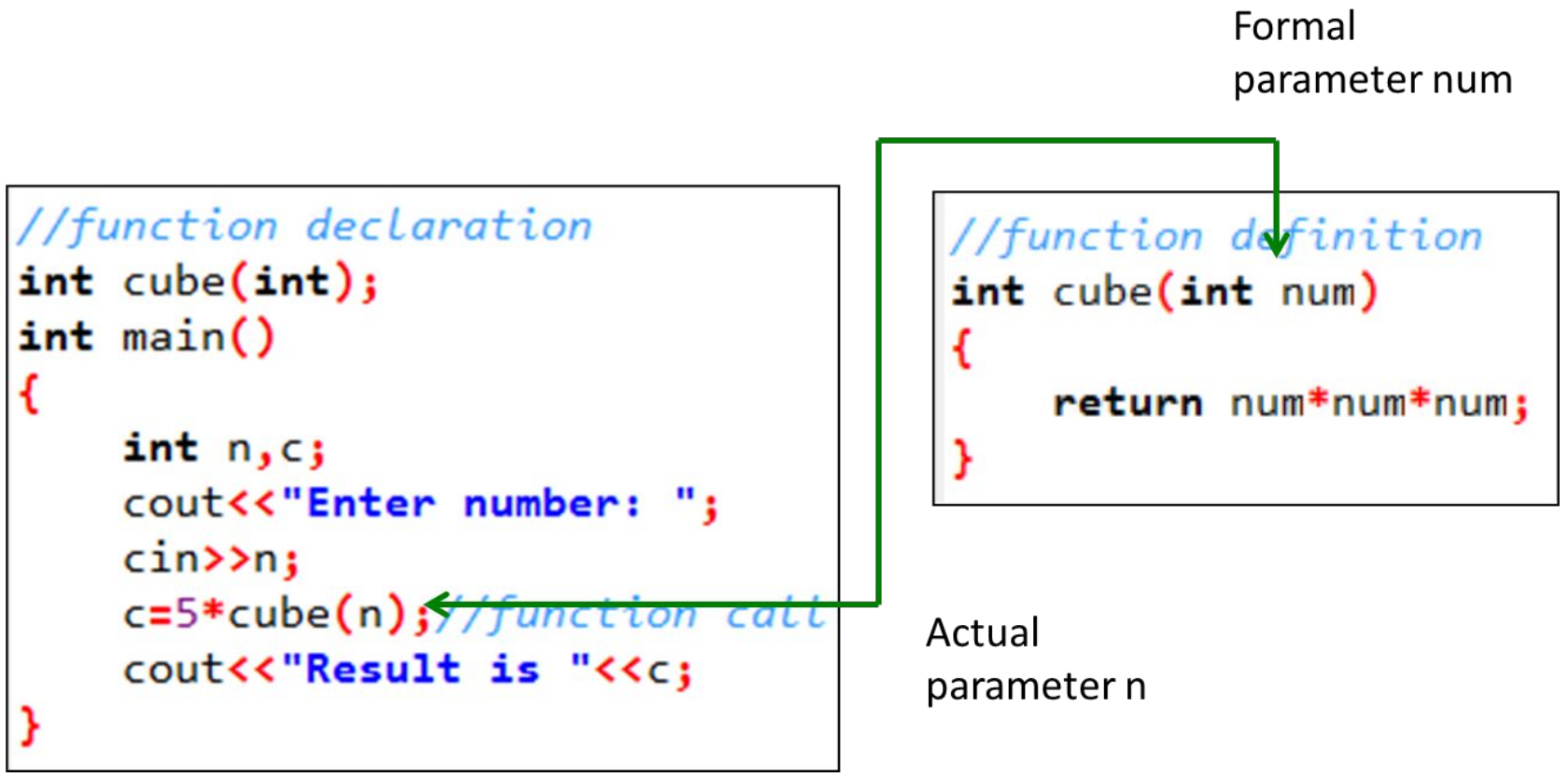
- return without value

  return;
- return with value

  return expression;



Example


Formal
parameter num
//function declaration
int cube(int);
int main()
{
int n,c;
cout<<"Enter number: ";
cin>>n;
c=5*cube(n);//function call
cout<<"Result is "<<c;
}
//function definition
int cube(int num)
{
return num*num*num;
}
Actual
parameter n

Continue Statement

The break statement terminates the block of code, in particular it terminates the execution of an iterative statement.

The continue statement forces the early termination of the current iteration to begin immediately the next iteration.

When continue statement is used inside the body of the loop it shifts the control to the start of the loop body.

When this statement is executed in the loop body, the remaining statements of current iteration are not executed.

The control directly moves to the next iteration.

```
int x;
for (x = 1; x<=5; x++)
{
        cout<<“Hello World”<<endl;
        continue;
        cout<<“Knowledge is power.”;
}
```

Questions?